

Micro-Manager from GUI to Python

Gabriel Selzer and Mark Tsuchida





Getting started with pymmcore-plus

From a uv-equipped terminal, find a good directory

```
cd mm-workshop-i2k-bina-2026 # Or wherever you want to run the workshop  
uv venv # Setup a virtual environment
```

Install pymmcore-plus (and run Python!)

```
uv pip install pymmcore-plus # Installs pymmcore-plus  
uv run mmcore use <path_to_your_micromanager> # Reuse your installation  
uv run python
```



Getting started with pymmcore-plus

`CMMCorePlus` is the core object

```
from pymmcore_plus import CMMCorePlus  
  
mmcore = CMMCorePlus()  
  
mmcore.getVersionInfo() # 'MMCore version 12.5.0' or similar
```



Getting Help

You can always call `help` on any function

```
help(mmcore.getVersionInfo)
```

Or you can go to the pymmcore-plus website:

<https://pymmcore-plus.github.io/pymmcore-plus/>



Initializing hardware

These can (and should) be reused in Python contexts!

```
mmcore.loadSystemConfiguration(<path_to_config_file>)  
mmcore.getLoadedDevices() # Should print out ('SimHub', 'SimCam', etc.)
```



Basic commands

Snap an image

```
mmcore.snap() # outputs a 2-dimensional NumPy array
```

Get the XY stage position

```
mmcore.getXYPosition() # outputs an (x, y) tuple
```

Set the XY stage position

```
mmcore.setXYPosition(100, 50) # call mmcore.getXYPosition() to see the change
```



Configuration Groups

Checking available configuration groups

```
mmcore.getAvailableConfigGroups() # should output ('Channel', 'Objective', etc.)
```

Checking available configuration group presets

```
mmcore.getAvailableConfigs('Objective') # should output ('100x 1.40NA Oil', etc.)
```

Set a configuration group preset

```
mmcore.setConfig('Objective', '100x 1.40NA Oil')
```



Multi-Dimensional Acquisitions

MDAs in pymmcore-plus run on MDASequences objects from useq:

```
from useq import MDASequences, GridRowsColumns

my_sequence = MDASequences(
    grid_plan=GridRowsColumns(
        rows=3, columns=3,
        fov_height=mmcore.getImageHeight() * mmcore.getPixelSizeUm(),
        fov_width=mmcore.getImageWidth() * mmcore.getPixelSizeUm(),
    )
) # Capture 9 fields of view, in a 3x3 grid
```



Multi-Dimensional Acquisitions

Execute an acquisition sequence

```
dest = <path to destination file>  
  
mmcore.run_mda(my_sequence, output=dest)
```

View acquisition events

```
mmcore.mda.events.frameReady.connect(lambda frame, event: print(event))
```

Mult

useq has **many** options!

- AutofocusPlan
- GridFromPolygon
- WellPlatePlan
- ZAboveBelow
- MultiPhaseTimePlan



...or you can write your own!

```
def run_mda(  
    self,  
    events: Iterable[MDAEvent],  
    *,  
    output: SingleOutput | Sequence[SingleOutput] | None = None,  
    block: bool = False,  
    dimension_overrides: dict[str, DimensionOverride] | None = None,  
    ) -> Thread:  
    """Run a sequence of [useq.MDAEvent][] on a new thread.
```

Define and

```
from useq import
```

```
my_sequence =
```

```
time_plan =
```

```
int
```

```
loc
```

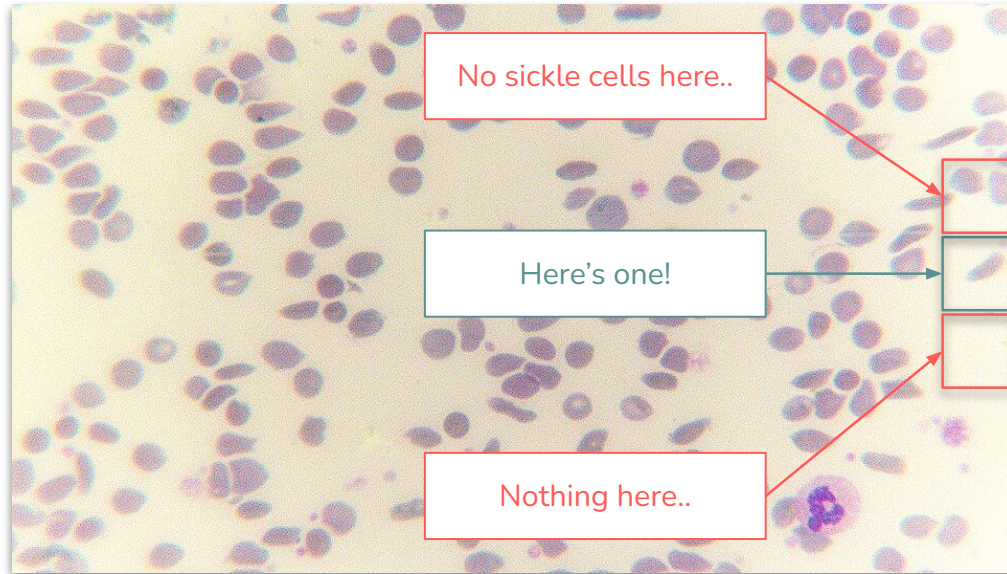
```
)
```

```
) # Capture
```

```
mmcore.run_mda(my_sequence, output=<path_to_another_dest_file>)
```

**Sometimes you really
want to write your own...**

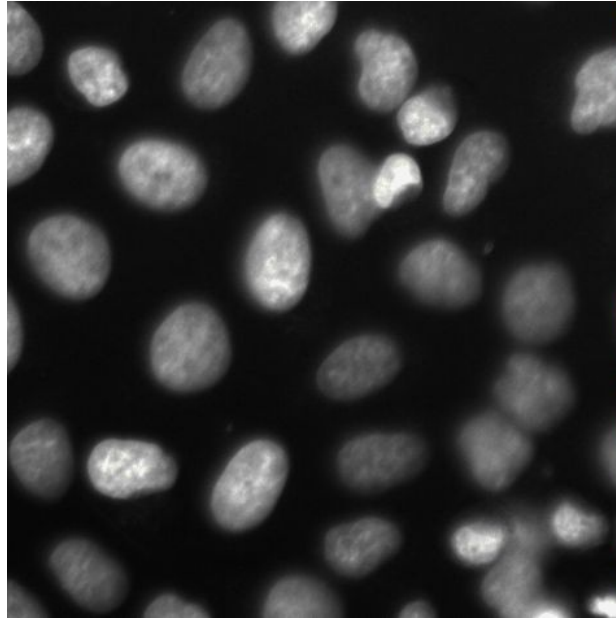
...when you can save frames



Sickle cells in blood

Image: Dr Graham Beards, CC BY-SA 4.0

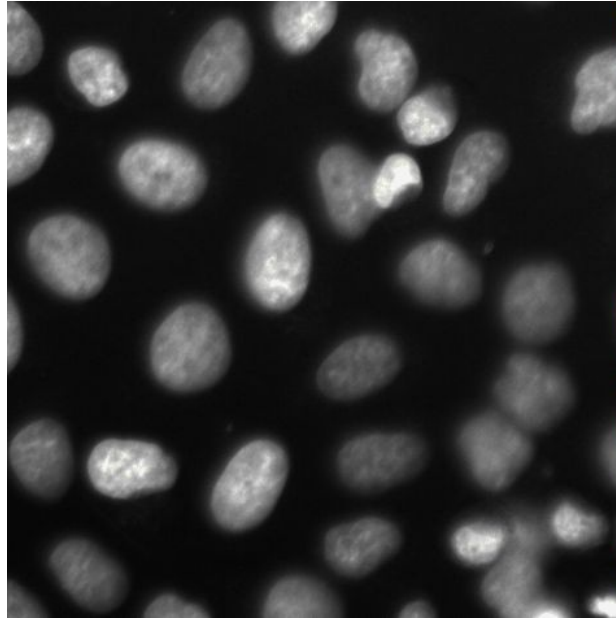
...when you can save frames



HeLa cell nuclei, imaged every 30 mins
Image: Ed Evans (LOCI)



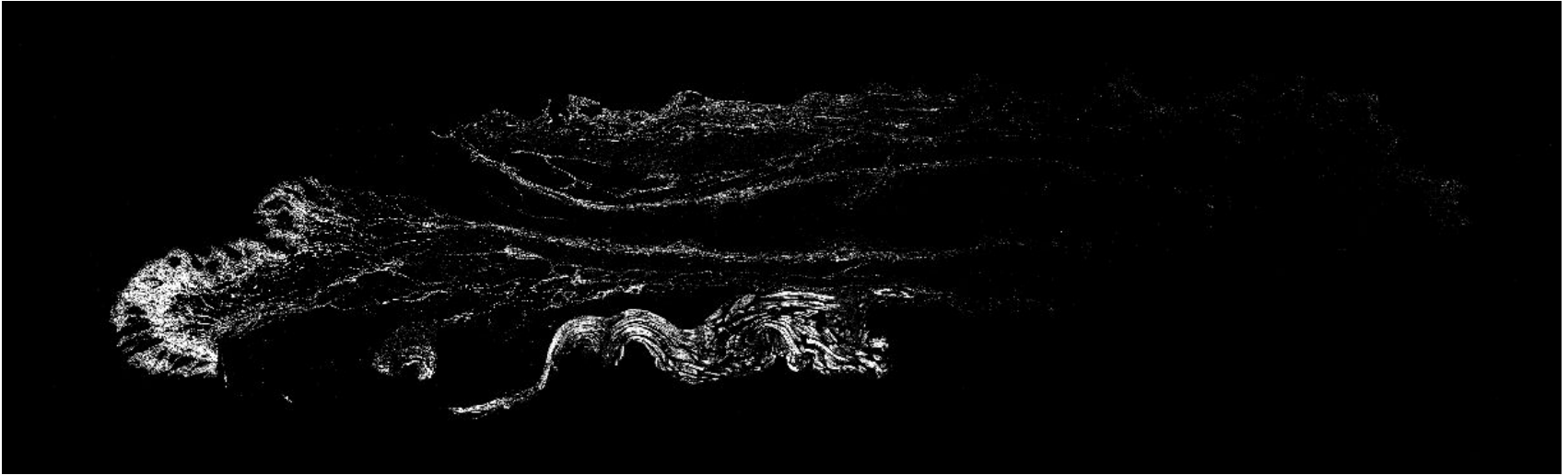
...to track moving targets



HeLa cell nuclei, imaged every 30 mins
Image: Ed Evans (LOCI)



...when you need autofocus



Mouse anal tissue, SHG
Image: Gabriel Selzer

**data-driven
acquisition needs
scripting**



Example

Identify sparse nuclei at
low resolution, then image
them at high resolution



Building our script, step-by-step

1. Initialize your `CMMCorePlus` object



Building our script, step-by-step

1. Initialize your CMMCorePlus object
2. Setup data viewing (optional)

This example uses **ndv**



<https://pyapp-kit.github.io/ndv/latest/>

Building our script, step-by-step

- 1.
- 2.
- 3.

This example uses **stardist**



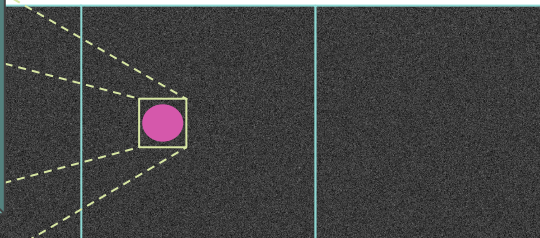
<https://stardist.net/>

via **cellcast**



<https://github.com/uw-loci/cellcast>

- 3a. Define the “steady state” behavior
- 3b. Implement target detection
- 3c. Define target response





Building our script, step-by-step

1. Initialize your CMMCorePlus object
2. Setup data viewing (optional)
3. Define your Iterable[MDAEvent]
4. Setup data writing

This example uses **ome-writers**

```
settings = AcquisitionSettings(  
    root_path=str(DATA_PATH / "high_res.ome.zarr"),  
    dimensions=(  
        Dimension(name="p", type="position", coords=coords),  
        Dimension(name="y", count=h, scale=self.px_size, unit="micrometer"),  
        Dimension(name="x", count=w, scale=self.px_size, unit="micrometer"),  
    ),  
    dtype="uint16",  
    overwrite=True,  
)  
with create_stream(settings) as stream:  
    for frame, _ in self._pois:  
        stream.append(frame)
```

Demo

<https://github.com/uw-loci/mm-workshop-i2k-bina-2026>